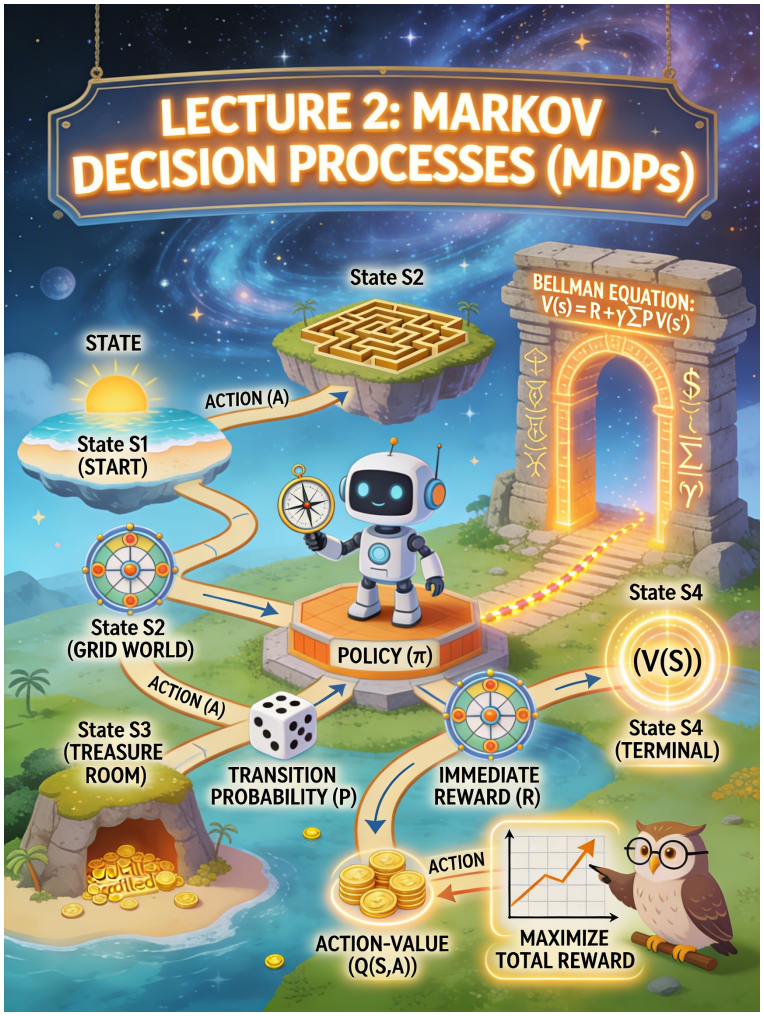
David Silver 强化学习课程笔记

Lecture2: Markov Decision Processes



SpiritsYouthHarmony

2026 年 4 月

本节的主题是马尔可夫决策过程 (Markov Decision Processes, MDPs)，首先要说明的是：MDPs 是对 RL 的环境建模。第一节课讲到，RL 的关键组成部分是智能体和环境，其中，智能体是我们要构建的算法，而环境就是通过 MDPs 进行建模的。实际上几乎所有强化学习问题都可以在某种程度上形式化为一个马尔可夫决策过程。

1 Markov Process

马尔可夫性："在已知当下的情况下，未来独立于过去"。如1，这是所有马尔可夫过程的核心点。

Definition (Markov Property)

A state S_t is Markov if and only if

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1, ..., S_t] \tag{1}$$

为了从数学上形式化定义马尔可夫过程，引入**状态转移**的概念。

Definition (state transition probability)

For a Markov state s and successor state s' , the state transition probability is defined by

$$\mathcal{P}_{ss'} = \mathbb{P}[S_{t+1} = s'|S_t = s] \tag{2}$$

Definition (state transition matrix)

State transition matrix $\mathcal{P}$ defines transition probabilities from all states s to all successor states s'

$$\mathcal{P} = \begin{matrix} \\ from \end{matrix} \begin{matrix} \begin{matrix} to \end{matrix} \\ \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \vdots \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \end{matrix} \tag{3}$$

where each row of the matrix sums to 1.

这个矩阵 $\mathcal{P}$ 给出了马尔可夫问题的完整结构：从任何状态，有多大的可能到达任何其他状态。
下面定义**马尔可夫过程**：

Definition (Markov Process)

A *Markov Process* (or *Markov Chain*) is a tuple $\langle \mathcal{S}, \mathcal{P} \rangle$

- $\mathcal{S}$ is a (finite) set of states
- $\mathcal{P}$ is a state transition probability matrix,

$$\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$$

简单来说，一个马尔可夫过程就是一个随机过程，是由一组具有马尔可夫性质的随机状态序列 $S_1, S_2, \dots$ 构成的。所以说，只需要**状态空间**和**转移概率矩阵**就能定义一个马尔可夫过程。

当然，举个例子会更容易理解，如图1展示了一个 Student Markov Chain，圆圈和方块代表不同的状态，箭头上标出的是转移概率。如果你在第一节课上，会有 0.5 的概率你会认真听完这节课然后接着上第二节课，然而，这节课可能在讲 MDP 这种令人头大的数学知识，有 0.5 的概率你会拿起手机开始浏览 Facebook，一旦开始浏览，很可能你会被吸引，0.9 的概率继续刷，不过如果你刷无聊了也有 0.1 的概率回来接着上课（笑）……

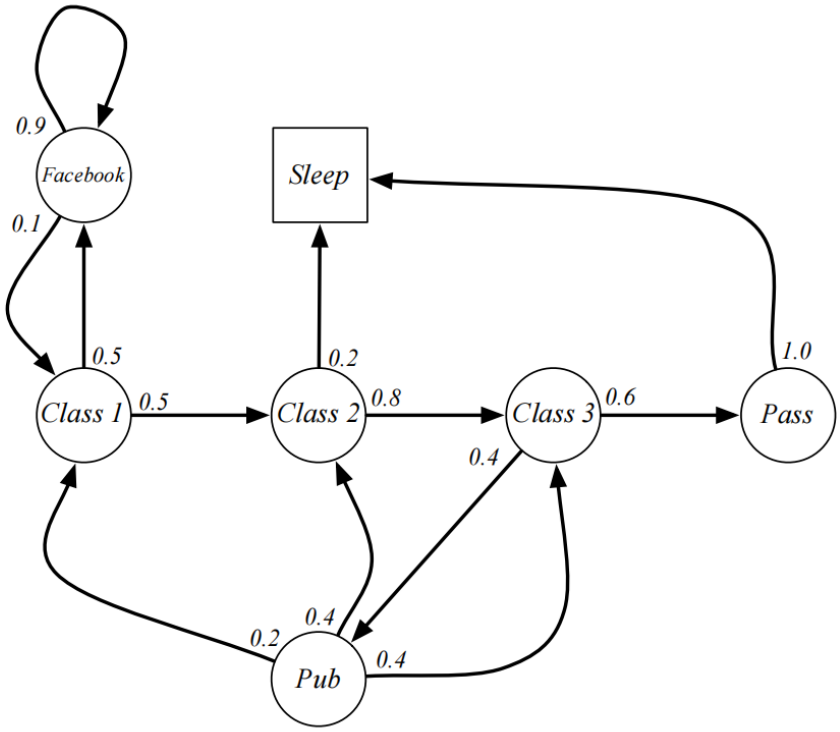


图 1: Student Markov Chain

我们可以把这个 Student Markov Chain 的转移矩阵写出来：

$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{array} \tag{4}$$

这样一个矩阵就已经完全建模了马尔可夫过程，把状态空间和转移概率都给涵盖了。

2 Markov Reward Process

Markov Process 太简单了，我们上节提到的强化学习中的奖励、动作等概念都没有，接下来我们会一点点让这个过程逐渐向强化学习的环境——MDPs 靠拢。
首先，引入**奖励**的概念，构造 Markov Reward Process：

Definition (Markov Reward Process)

A *Markov Reward Process* is a tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{P}$ is a state transition probability matrix,

$$\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$$

- $\mathcal{R}$ is a reward function, $\mathcal{R}_s = \mathbb{E}\left[R_{t+1} \mid S_t = s\right]$
- γ is a discount factor, $\gamma \in [0, 1]$

相比于 Markov Process，Markov Reward Process 就是在已有状态空间和转移概率矩阵的基础上，添加一个奖励函数 $\mathcal{R}$ 和一个折扣因子 γ。

- 根据奖励函数的定义，$\mathcal{R}$ 告诉我们，如果我们在某个时刻处于 s 这个状态，我们能从这个状态得到多少奖励，并且是即时奖励。在强化学习中，我们最终的目标就是要最大化这些奖励的和。

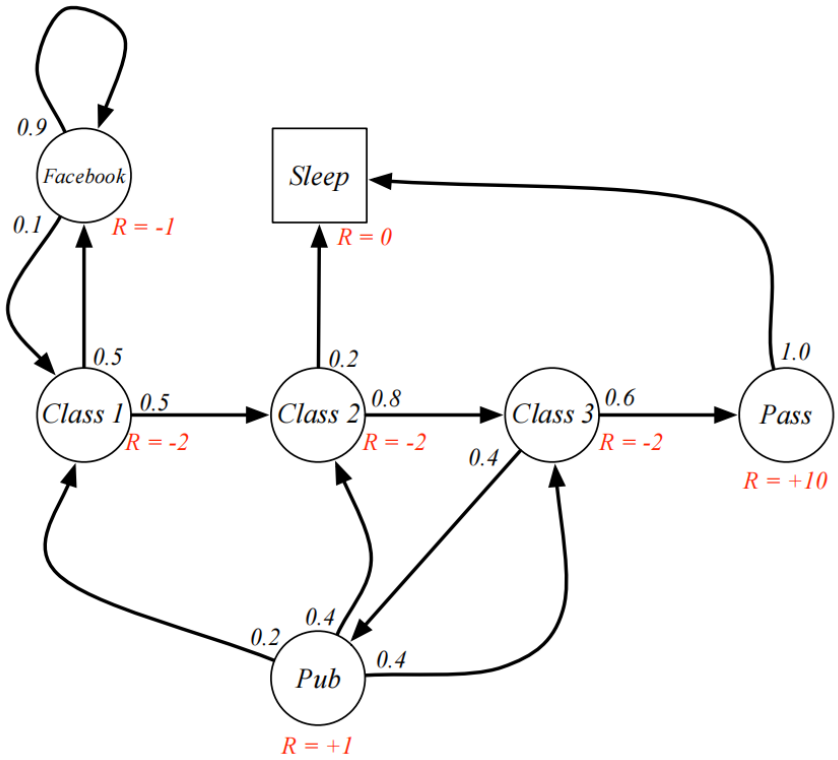


图 2: Student MRP

如图2，加入了奖励之后的 Student Markov Chain 构成一个 Student MRP，你并不喜欢上课，每节课都会得到一个-2 的奖励，但当你上完三节课通过考试时，你会得到一个 +10 的大额奖励；而如果你上课期间去刷 Facebook，会得到一个-1 的奖励……
如果你按照“Class1→Class2→Class3→Pass”这样一条路径进行状态转移，你最终会得到 $-2-2-2+10=4$ 的奖励值。

下面来聊聊折扣因子

- 折扣因子 γ 是一个非常特别的量，因为在讲强化学习的基本概念时我们从未提到过这个量。然而，在几乎所有的 RL 问题中，这个折扣因子都是不可或缺的。

下面，我们定义一个量——Return，中文可译为“回报”。

Definition (Return)

The *return* G_t is the total discounted reward from time-step t.

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{5}$$

这里我们用大写字母 G 来表示，其实是因为这个量正是我们希望最大化的目标（Goal），而折扣因子在其中是一个用来控制未来时间步奖励的量。需要注意的是，回报是一个随机量，是从 MRP 过程中抽取的一条路径上的折扣奖励的总和。

简单来说，这个因子描述的是“我们有多关心未来得到的奖励”：

- γ close to 0 leads to “myopic” evaluation
    - 也就是说将当前时间步之后的任何奖励都归零，只看第一个奖励。
- γ close to 1 leads to “far-sighted” evaluation
    - 这代表最大程度的“远见”，关心未来的所有奖励。

之所以会有这么个东西，主要有以下几个原因：

- 把奖励折现从数学上来说是可行的，且避免出现无限奖励的情况。
- 未来的信息有更大的不确定性。不管是 MRP 还是后面的 MDP，本质上都是为了建模 RL 的环境，然而，并不存在一个完美的 model，只是说从经验来看，越远的未来越不确定，加上这个折扣因子会更加贴近实际。举个例子，如果你现在拿到一块金子，你按照当下的金价把它卖了，即可可以收益，可是如果有人告诉你十年后这块金子有 50% 的概率涨价为现在的 2 倍，那么你会如何抉择？如果你对这件事有足够的信心，也许你会等，但相比于眼下即刻可得的收益，是否要真的等上十年？
- 研究显示，动物和人类也表现出对即时奖励的偏好。因此，我们可以把折扣因子看作是一个关于生物决策的认知模型。

刚刚说到，回报是一个随机量，是某个路径奖励的累计和，而我们真正关心的是期望意义的回报，这才能衡量出某个状态真正的价值。

下面，定义**价值函数（value function）**

Definition (state value function for MRP)

The *state value function* $v(s)$ of an MRP is the expected return starting from state s

$$v(s) = \mathbb{E}[G_t \mid S_t = s] \tag{6}$$

价值函数衡量的是一个状态的长期价值，是从某个状态出发的所有回报的期望值。之所以使用期望，是因为环境是随机的。状态的演变可能在一个回合中走向一个方向，在下一个回合中走向另一个方向。我们关心的是所有这些不同回合上的期望回报。
还是以图2中的 Student MRP 为例，假设我们要计算“Class1”这个状态的 value function，那我们需要采样从这个状态出发的所有路径，计算回报并按照概率加权求和。

- “Class1→Class2→Class3→Pass→Sleep”
  - $G_1 = -2 - 2 * \gamma - 2 * \gamma^2 + 10 * \gamma^3$
- “Class1→Facebook→Facebook→Class1→Class2→Sleep”
  - $G_1 = -2 - 1 * \gamma - 1 * \gamma^2 - 2 * \gamma^3 - 2 * \gamma^4$
- ……

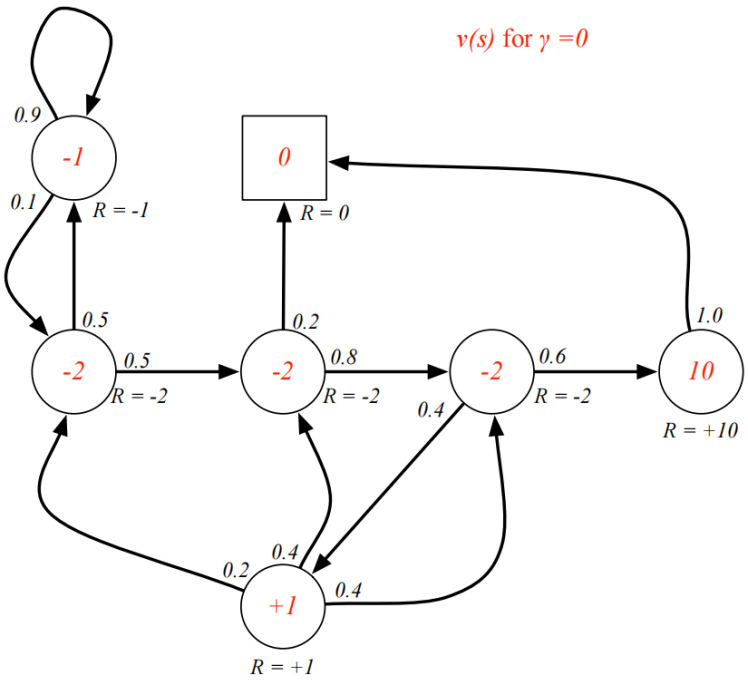


图 3: State-Value Function for Student MRP:$\gamma = 0$

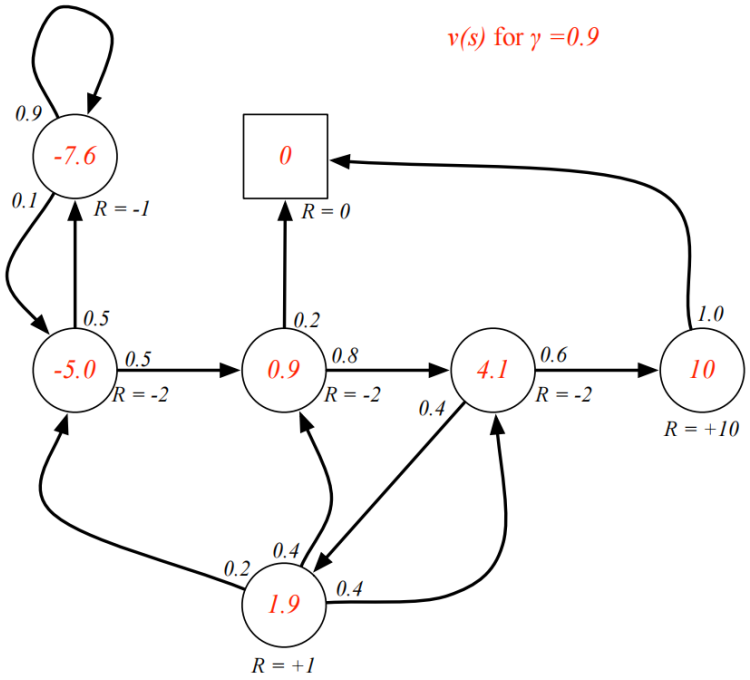


图 4: State-Value Function for Student MRP:$\gamma = 0.9$

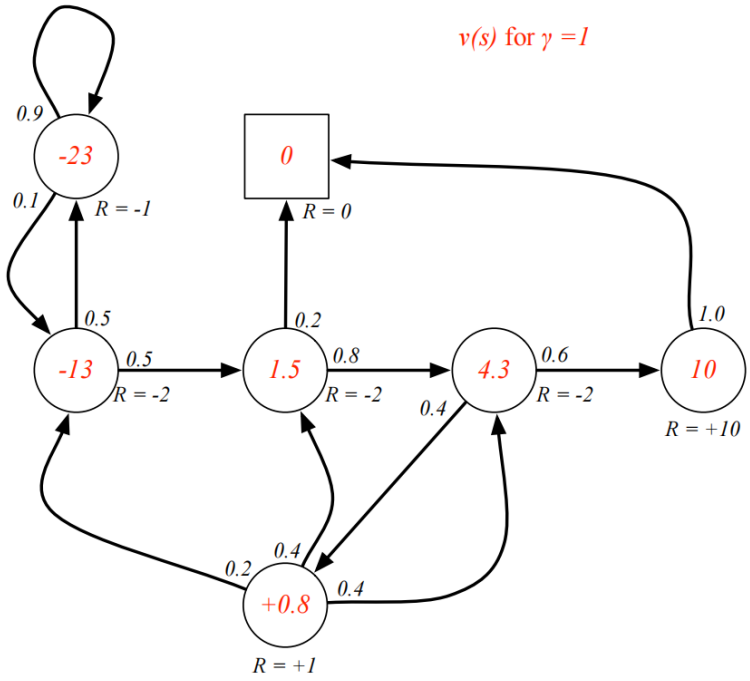


图 5: State-Value Function for Student MRP:$\gamma = 1$

如图3、图4、图5，分别展示了 $\gamma = 0$、$\gamma = 0.9$、$\gamma = 1$ 时 Student MRP 的 state value 值。（至于怎么计算的，就是后话了）

有了价值函数的定义，我们接下来讨论强化学习的基本关系，也就是大名鼎鼎的 **Bellman Equation（贝尔曼方程）**

价值函数可以被分解为两部分：

- 即时奖励 R_{t+1}
- 后继状态的折扣价值 $\gamma v(S_{t+1})$

$$
\begin{aligned}
v(s) &= \mathbb{E}\left[G_t \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma\left(R_{t+2} + \gamma R_{t+3} + \ldots\right) \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma G_{t+1} \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]
\end{aligned} \tag{7}
$$

这里头 $\mathbb{E}\left[G_{t+1} \mid S_t = s\right] = \mathbb{E}\left[\mathbb{E}\left[G_{t+1} \mid S_{t+1}, S_t = s\right] \mid S_t = s\right]$（全期望公式），而由于马尔可夫性，$\mathbb{E}\left[G_{t+1} \mid S_{t+1}, S_t = s\right] = \mathbb{E}\left[G_{t+1} \mid S_{t+1}\right] = v(S_{t+1})$，所以说 $\mathbb{E}\left[G_{t+1} \mid S_t = s\right] = \mathbb{E}\left[v(S_{t+1}) \mid S_t = s\right]$

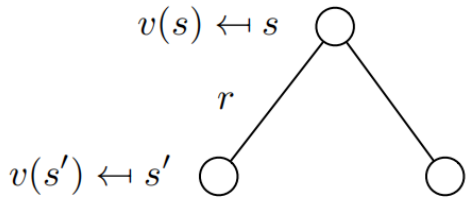


图 6: backup diagram

怎么理解这个公式？如6，这叫做 backup diagram，展示了 one-step lookahead 的过程。简单来说，状态 s 对应其价值 v(s)，如果我们以此为根节点，向下走到 s'（后继状态），对应价值 v(s')，沿途我们会获得即时奖励 r，按照概率把所有的 r 和 v(s') 平均下来，就是 s 的价值。
因此，v(s) 可以写成：

$$
v(s) = \mathcal{R}_s + \gamma \sum_{s'} \mathcal{P}_{ss'} v(s') \tag{8}
$$

如图7，以“Class3”这个状态为例，我们来验证一下 Bellman Equation，v(Class3)=4.3，即时奖励为-2，有 0.4 的概率转移至“Pub”状态，有 0.6 的概率转移至“Pass”状态，所以 $v(\text{Class3}) = -2+0.6*10+0.4*0.8 \approx 4.3$，显然，Bellman Equation 是成立的。

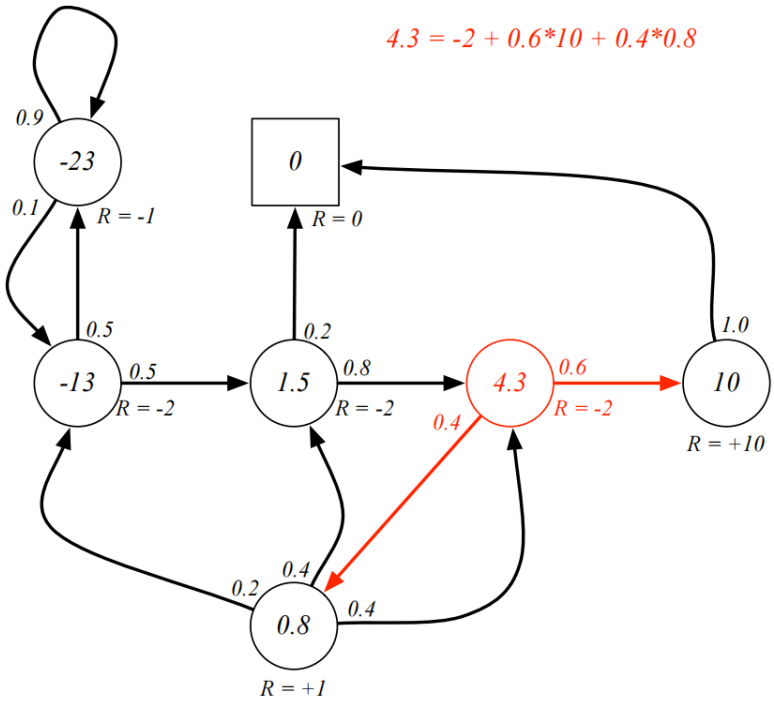


图 7: Bellman Equation for Student MRP

使用矩阵和向量的形式，Bellman Equation 可以写成：

$$v = \mathcal{R} + \gamma \mathcal{P} v \tag{9}$$

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \vdots \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

需要特别强调的是，Bellman Equation 是我们后续所有算法的基础，因为要求 state value，本质上就是在求解 Bellman Equation。

3 Markov Decision Process

Markov Reward Process 在 Markov Process 的基础上添加了奖励的概念，但是对 agent 来说，这个过程还是没有自主权，只能被动地进行状态转移。下面，我们把**动作**的概念加入，构造**马尔可夫决策过程**：

Definition (Markov Decision Process)

A *Markov Decision Process* is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix,

$$\mathcal{P}^{a}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s, A_t = a\right]$$

- $\mathcal{R}$ is a reward function, $\mathcal{R}^{a}_{s} = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right]$
- γ is a discount factor, $\gamma \in [0, 1]$

现在，转移概率依赖于 agent 采取的动作，也就是说在 MDP 中，agent 有了选择的权力。如图8所示，Student MDP 中，圆形和矩形框表示状态，箭头上的红字表示 agent 在相应状态可选的 action，这里需要注意，原来的 “Pub” 状态不存在了，黑色点不表示状态，只是代表选择 “Pub” 这个 action 时，状态转移是 stochastic 的，其他的所有动作都会导致一个 deterministic 的状态转移。

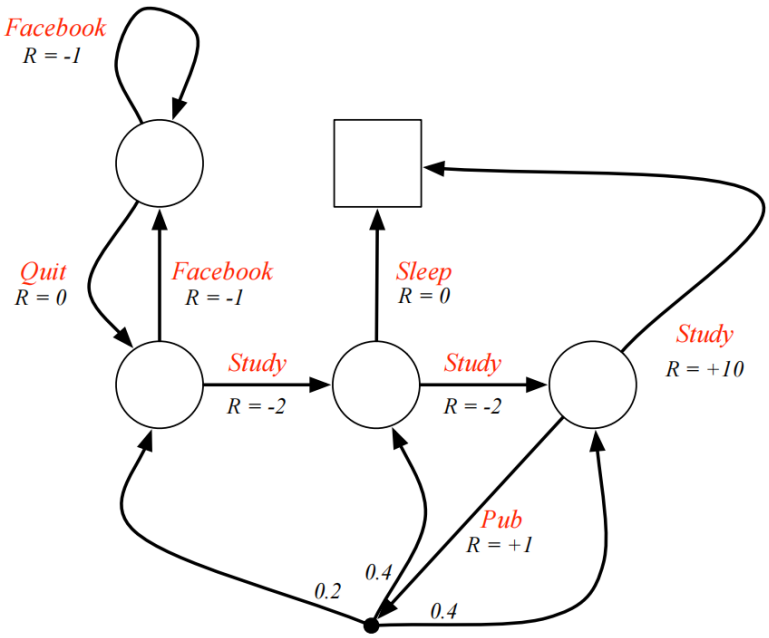


图 8: Student MDP

有了 MDP，对于 agent 来说，其可以通过选择动作来实现最大化累计回报的目标了。为了从数学上对这件事形式化，我们先引入一个重要的概念——Policy（策略）：

Definition (Policy)

A *policy* π is a distribution over actions given states,

$$\pi(a|s) = \mathbb{P}\left[A_t = a \mid S_t = s\right] \tag{10}$$

策略是给定状态下动作的分布。换句话说，如果 agent 处于某个状态 s，这个分布基本上给出了一个映射——比如说，如果 agent 处于状态 s，就以某个概率向左走，以某个概率向右走。

在 MDP 中，策略仅依赖于当前状态（马尔可夫性），不依赖于历史。并且，我们只考虑平稳策略，不考虑随时间变化的策略。在 MDP 的每个时间步，agent 都使用相同的策略，唯一依赖的是当前所处的状态，而不是发生的时间步。
在 MRP 中，我们定义了 state value function 用以描述状态的价值，在 MDP 中，我们也需要这个概念，只不过此时 agent 可以选择动作，所以要在策略 π 的约束下计算 state value：

Definition (state value fucntion for MDP)

The *state-value function* $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t \mid S_t = s\right] \tag{11}$$

当然，在 MDP 中，除了 state value，我们还关注 action value：

Definition (action value fucntion for MDP)

The *action-value function for MDP* $q_\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi\left[G_t \mid S_t = s, A_t = a\right] \tag{12}$$

action value 是在告诉我们，agent 在特定状态 s 采取特定动作 a 到底有多好。换句话说，对于给定的策略（给定的行为方式），我们想知道：如果 agent 处于状态 s 并且采取动作 a，在采取该动作之后，将获得多少期望回报？

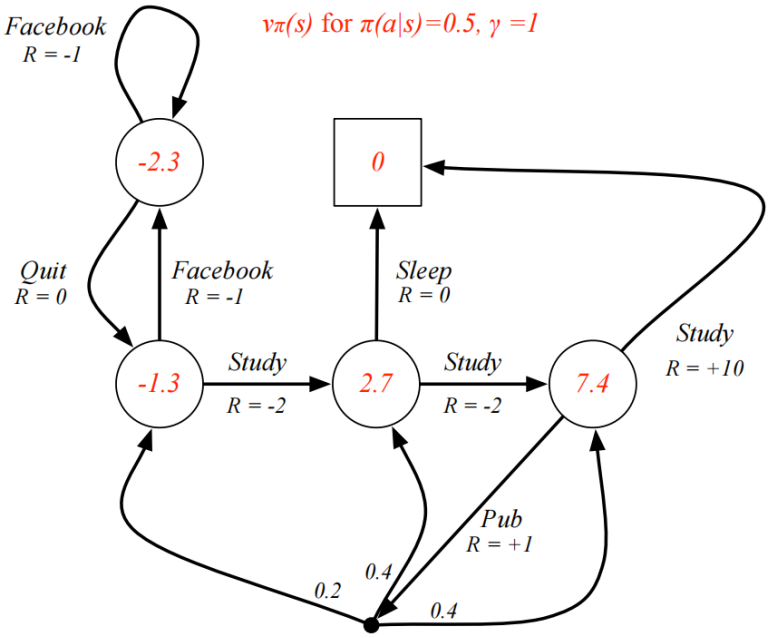


图 9: state value function for Student MDP

对于 Student MDP，每个状态都有两个可选动作，假设采取一种均匀策略，也就是在任何一个状态 s 中选择其可选的两种动作的概率都是 0.5，此时的 state value 如图9。
之前说过，怎么计算这些 state value 是后话，但是必须得把 Bellman Equation 写出来，因为求 state value 就是求解 Bellman Equation。

对于 MDP 的 Bellman Equation，核心思想仍然是：价值函数可以分解为即时奖励加上后继状态的折扣价值

Bellman Expectation Equation

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right] \tag{13}$$

$$q_\pi(s,a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right] \tag{14}$$

下面我们还是用 backup diagram 来理解：

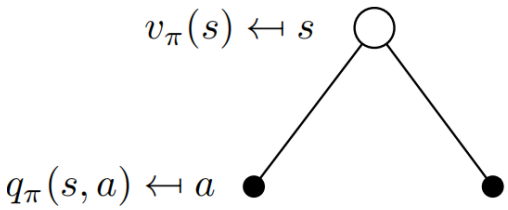


图 10: backup diagram

如图10，状态 s 用空心圆圈表示，其对应状态价值 $v_\pi(s)$ ，在该状态，agent 有若干动作可选，所以 look-ahead one-step 就到了动作节点（黑色实心圆圈表示），每个动作 a 对应一个动作价值 $q_\pi(s,a)$，那么，根节点 s 的状态价值应当等于所有动作价值按照策略 π 加权的结果，如式15所示。

$$v_\pi(s) = \sum_{a\in\mathcal{A}} \pi(a|s) q_\pi(s,a) \tag{15}$$

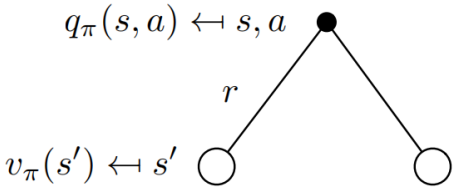


图 11: backup diagram

如图11，这回从动作节点出发，采取动作后会得到一个即时奖励，此外，根据转移概率，有可能会转移到不同的状态，所以子节点是不同的状态。那么，动作价值就应该是即时奖励加上不同的后继状态价值按照转移概率加权求和的结果，如式16所示。

$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s'\in\mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \tag{16}$$

很显然，上面这两张图可以拼起来，做一个 two-step look-ahead

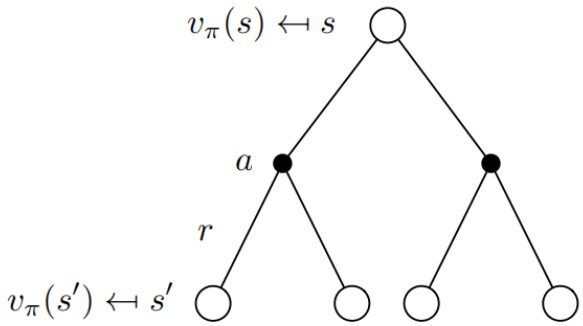


图 12: backup diagram

如图12，把状态作为根节点，two-step look-ahead 之后到达后继的状态节点，根据前述分析，$v_\pi(s)$ 可以写成如式17所示的形式，这是一种 bootstrapping 的形式，也是我们后续进行求解时用到的 Bellman Expectation Equation。

$$v_\pi(s) = \sum_{a\in\mathcal{A}} \pi(a|s) q_\pi(s,a) = \sum_{a\in\mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s'\in\mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right) \tag{17}$$

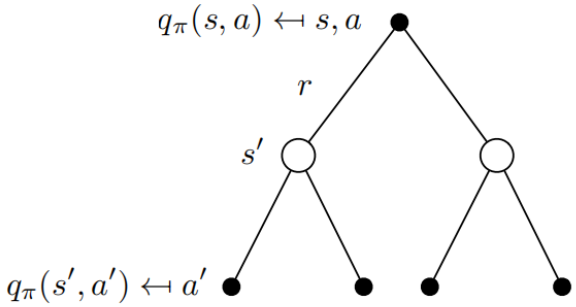


图 13: backup diagram

同理，如图13，把动作作为根节点，two-step look-ahead 之后到达后继的动作节点，根据前述分析，$q_\pi(s,a)$ 可以写成如式18所示的形式。

$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s',a') \tag{18}$$

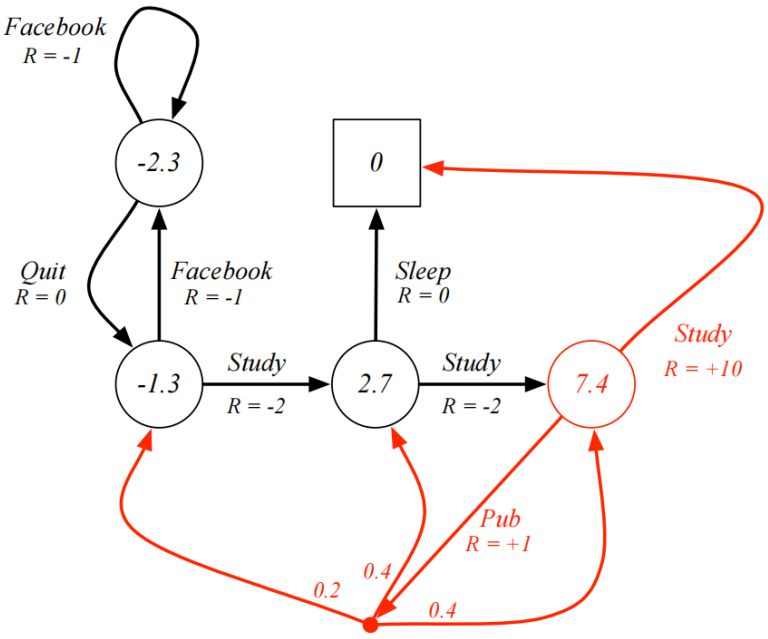


图 14: Bellman Expectation Equation in Student MDP

如图14，以最后这个圆圈表示的状态为例，验证一下 Bellman Expectation Equation：

$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$

$7.4 = 0.5 * (10 + 0) + 0.5 * (1 + 0.2 * (-1.3) + 0.4 * 2.7 + 0.4 * 7.4)$

前面这部分介绍了 MDP、value function 以及 Bellman Equation，但我们最关心的问题是：如何在 MDP 中找到最优的一种行为，或者说，如何找到 MDP 的最优解?

Definition (optimal value fucntion for MDP)

The *optimal state-value function* $v_*(s)$ is the maximum value function over all policies

$$v_*(s) = \max_\pi v_\pi(s) \tag{19}$$

The *optimal action-value function* $q_*(s,a)$ is the maximum action-value function over all policies

$$q_*(s,a) = \max_\pi q_\pi(s,a) \tag{20}$$

- $v_*(s)$：从状态 s 出发，遵循最优策略所能获得的期望累积折扣回报。
- $q_*(s,a)$：从状态 s 出发，先执行动作 a，之后遵循最优策略所能获得的期望累积折扣回报。

可以这么说，只要求解出最优价值函数，MDP is solved.

当然，我们关心的是最优策略，在 MDP 中什么是最好的行为方式？我们先定义一个偏序关系：

$$\pi \geq \pi' \text{ if } v_\pi(s) \geq v_{\pi'}(s), \forall s \tag{21}$$

存在以下定理：

Theorem

For any Markov Decision Process

- There exists an optimal policy π_* that is better than or equal to all other policies, $\pi_* \geq \pi, \forall \pi$
- All optimal policies achieve the optimal value function,

$$v_{\pi_*}(s) = v_*(s)$$

- All optimal policies achieve the optimal action-value function,

$$q_{\pi_*}(s,a) = q_*(s)$$

这个定理告诉我们，最优策略一定存在，而其求解方式就是：解出 q_*，然后选择能带来最大 q_* 的动作。换句话说，在每个状态 s 下，agent 要做的就是以概率 1 选择动作 a，使得 $q_*(s,a)$ 最大，这也是刚刚为什么说“只要求解出最优价值函数，MDP is solved”。

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\text{argmax}}\ q_*(s,a) \\ 0 & \text{otherwise} \end{cases} \tag{22}$$

同理，我们也可以按照 look-ahead 的方式找出 v_* 和 q_* 的关系，这里直接给出结论：

Bellman Optimality Equation

$$v_*(s) = \max_a q_*(s,a) \tag{23}$$

$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \tag{24}$$

$$v_*(s) = \max_a \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \right) \tag{25}$$

$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a') \tag{26}$$

拿第一个式子来说，如图15，Bellman Optimality Equation 是在回答：从状态 s 开始，处于某个状态的最优价值 $v_*(s)$ 是多少？考虑可能采取的每个动作，每个动作会到达对应的动作节点。当 agent 到达每个动作节点时，也可以查看动作价值，然后取它们中的最大值就代表着这个状态有多好，它就是所有 q 值的最大值。

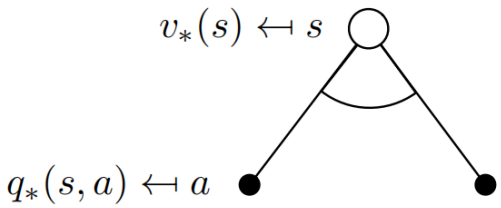


图 15: backup diagram

后续课程本质上也都是在求解 Bellman Optimality Equation，只是采用不同的方法：

- Value Iteration
- Policy Iteration
- Q-learning
- Sarsa
- ……

参考文献

[1] David Silver. *Lectures on Reinforcement Learning*. URL: `https://www.davidsilver.uk/teaching/`, 2015.